TDMA SCHEDULER

Detailed Technical Project Documentation

Topology-Aware Time Division Multiple Access Scheduling with EMANE Integration

Project Type	Network Simulation / TDMA Scheduling
Simulation Platform	EMANE 1.5.3
Implementation	Python + NetworkX
Test Topology	16-node 4×4 grid
Configured Radio Range	500 m
Optimized Frame	9 unique timeslots
Verification	16 NEMs + packet-level connectivity test

Formal technical documentation and implementation record

Document Control

Item	Details
Project name	TDMA Scheduler
Document	Detailed Technical Project Documentation
Environment	WSL2 / Ubuntu 22.04
EMANE version	1.5.3
Validated topology	16 nodes, 4×4 grid
Validated schedule	1 frame, 9 slots, 1000 µs slot duration
Validation	Conflict-free schedule and 5/5 packet connectivity test

Contents

1. Executive Summary
2. Introduction and Background
3. Problem Statement
4. Objectives
5. Scope and Assumptions
6. System Architecture
7. TDMA Scheduling Methodology
8. Topology and Conflict Model
9. Implementation Details
10. EMANE Integration
11. Configuration Files
12. Generated Schedule
13. Installation and Execution
14. Verification and Results
15. WSL2 Considerations
16. Limitations
17. Future Scope
18. Conclusion
19. Appendix

1. Executive Summary

- The TDMA Scheduler is a topology-aware scheduling project that generates a conflict-free Time Division Multiple Access schedule for a 16-node wireless topology.
- The implementation processes a topology input and configured radio range, derives scheduling relationships, and assigns transmitting nodes to timeslots while respecting the project's conflict model.
- For the demonstrated 16-node test case at a 500 m configured radio range, the scheduler produced an optimized frame of 9 unique timeslots and reported the schedule as conflict-free.
- The generated schedule was integrated with EMANE 1.5.3 using the TDMA Event Scheduler Radio Model and 16 EMANE Network Emulation Modules (NEMs).
- The EMANE environment created virtual interfaces emane1 through emane16 with addresses 172.30.1.1 through 172.30.1.16.
- The TDMA event schedule was published successfully. A packet-level connectivity test returned 5 packets transmitted, 5 received, and 0% packet loss.

End-to-end workflow: topology input → scheduling → TDMA XML → EMANE platform → event publication → virtual interfaces → packet-level validation.

2. Introduction and Background

2.1 TDMA concept

Time Division Multiple Access divides a shared communication medium into discrete time intervals. Nodes receive planned transmission opportunities, allowing controlled access instead of arbitrary contention.

2.2 Project motivation

A one-node-per-slot schedule is simple but may waste capacity. If multiple nodes can transmit simultaneously without violating the modeled conflict constraints, they can reuse the same slot. The scheduler therefore seeks compact, conflict-free slot assignments.

2.3 Role of EMANE

EMANE provides the network emulation environment used to load and exercise the generated TDMA schedule. It is the validation platform, while the scheduling logic is implemented in Python.

3. Problem Statement

Design and implement a topology-aware TDMA scheduler that accepts a multi-node topology and radio range, determines scheduling conflicts, assigns compatible transmitters to common timeslots, and produces a schedule suitable for EMANE integration.

4. Project Objectives

- Develop a reusable Python TDMA scheduling workflow.
- Represent the wireless topology programmatically.
- Use the configured radio range in the conflict model.
- Group compatible transmitters into common slots.
- Reduce frame length subject to the modeled constraints.
- Generate a node-to-slot assignment and structural schedule matrix.
- Export a TDMA schedule suitable for EMANE.
- Configure and launch 16 EMANE NEMs.

- Publish the schedule through the EMANE event service.
- Verify interfaces, addresses, schedule loading, and packet-level connectivity.

5. Scope and Assumptions

- 16-node 4×4 grid topology.
- 500 m configured radio range for the demonstrated run.
- TDMA frame optimization and conflict-free slot assignment.
- EMANE 1.5.3 integration using the TDMA Event Scheduler Radio Model.
- Virtual transport and packet-level validation under WSL2.
- The 500 m value is a simulation configuration, not a physical radio measurement.
- The packet test validates the configured virtual path and is not a physical wireless propagation or hardware test.

6. System Architecture

Layer	Responsibility	Artifact
Input	Read topology and parameters	sample_input.json
Scheduling	Build relationships and assign slots	tdma_scheduler.py
Output	Print assignment and matrix	run_from_file.py
EMANE	Configure NEMs, MAC/PHY, transport	emane/*.xml
Validation	Publish schedule and test connectivity	event publisher + ping

Data flow

Topology JSON → **Python scheduler** → **conflict-aware slot assignment** → **9-slot TDMA schedule** → **EMANE configuration** → **event publication** → **packet test**

7. TDMA Scheduling Methodology

7.1 Input processing

The demonstrated scheduler execution was:

```
python3 run_from_file.py sample_input.json --range 500
```

7.2 Topology and range

The topology and configured range provide the basis for deciding which nodes may share a transmission slot under the scheduler's conflict model.

7.3 Slot assignment

Compatible transmitters are grouped into the same slot. Nodes that would violate the modeled constraints are assigned to different slots. The output is a sequence of slots containing one or more transmitters.

7.4 Optimization result

The demonstrated run processed 16 nodes and produced 9 unique timeslots. The scheduler explicitly reported the resulting schedule as conflict-free.

8. Topology and Conflict Model

The project documentation specifies a 4×4 grid with 16 nodes and a 500 m radio range. The scheduler output shows the resulting spatial reuse rather than requiring one slot per node.

Parameter	Value
Nodes	16
Topology	4×4 grid
Radio range	500 m
Optimized frame	9 slots
Frames	1
Slot duration	1000 µs
Slot overhead	0 µs
Bandwidth	1 MHz
Frequency	2.4 GHz
Data rate	1 Mbit/s

9. Implementation Details

9.1 Main project files

File	Purpose
tdma_scheduler.py	Core scheduling implementation.
run_from_file.py	Runs the scheduler from an input file.
analyze_topology.py	Topology analysis utility.
test_scheduler.py	Scheduler tests.
validation_tests.py	Validation tests.
sample_input.json	Sample topology input.
spatial_reuse_test.json	Spatial reuse test input.
hidden_terminal_test.json	Hidden-terminal test input.
emane/platform-16nem.xml	16-NEM platform definition.
emane/tdma-schedule.xml	TDMA schedule delivered to EMANE.
emane/tdmaradiomodel.xml	TDMA radio-model configuration.
emane/transvirtual.xml	Virtual transport configuration.
emane/nem/*.xml	Per-NEM definitions.

9.2 Environment

- WSL2 / Ubuntu 22.04.
- EMANE 1.5.3 verified with emane --version.
- Python 3 used for scheduler and EMANE event publication.

- NetworkX is required by the scheduler.
- Linux networking commands were used for interface and address verification.

10. EMANE Integration

10.1 Platform

The platform XML defines the 16-NEM environment, control endpoint, event service multicast settings, OTA multicast settings, and per-NEM definitions.

10.2 NEM configuration

Each NEM uses EMANE virtual transport, a unique virtual interface, a unique 172.30.1.x address, the TDMA radio model, and PHY parameters. The demonstrated NEMs use subid 7.

NEMs	Devices	Addresses	subid
1–16	emane1 … emane16	172.30.1.1 … 172.30.1.16	7

10.3 TDMA radio model

tdmaradiomodel.xml uses the tdmaeventschedulerradiomodel library and specifies queue, fragmentation, aggregation, neighbor-metric, and PCR parameters.

10.4 Virtual transport

transvirtual.xml uses the transvirtual transport and Linux TUN device path /dev/net/tun. The demonstrated environment successfully created the 16 virtual EMANE interfaces after TUN access was available.

11. Configuration Files

11.1 Platform structure

```
<platform name="TDMA 16 Node Test Platform">
  <param name="controlportendpoint" value="0.0.0.0:47000"/>
  <param name="eventservicegroup" value="224.1.2.8:45703"/>
  ...
</platform>
```

The NEM definition files were copied into the platform directory because the EMANE platform DTD rejected a path in the definition attribute. The working configuration references tdmanem-1.xml through tdmanem-16.xml.

11.2 TDMA schedule structure

```
<structure slotduration="1000" slotoverhead="0" frames="1" slots="9" bandwidth="1M"/>
```

The schedule contains one frame, nine slots, 1000 µs slot duration, zero slot overhead, and 1 MHz bandwidth. The multiframe specifies 2.4 GHz frequency, zero power, class 0, and 1 Mbit/s data rate.

12. Generated Schedule

12.1 Node-to-slot assignment

Node	Slot	Node	Slot
Node_01	8	Node_09	7
Node_02	4	Node_10	2
Node_03	5	Node_11	3
Node_04	8	Node_12	7
Node_05	6	Node_13	8
Node_06	0	Node_14	4
Node_07	1	Node_15	5
Node_08	6	Node_16	8

12.2 Slot-wise schedule

Slot	Transmitting NEMs
0	6
1	7
2	10
3	11
4	2, 14
5	3, 15
6	5, 8
7	9, 12
8	1, 4, 13, 16

The multi-node slots demonstrate spatial reuse in the generated schedule. The scheduler output reported the complete frame as conflict-free.

13. Installation and Execution Procedure

13.1 Verify EMANE

```
emane --version
```

Expected verified version: 1.5.3.

13.2 Verify Python package path

The EMANE Python package was installed under /usr/local/lib/python3/dist-packages and the environment was configured so that Python could import emane. The event utility then responded to --help successfully.

13.3 Run the scheduler

From the project root:

```
python3 run_from_file.py sample_input.json --range 500
```

13.4 Start EMANE

```
sudo emane emane/platform-16nem.xml
```

13.5 Publish the TDMA schedule

```
emaneevent-tdmaschedule emane/tdma-schedule.xml
```

13.6 Verify interfaces

```
ip -br link | grep emane
ip -br addr | grep emane
```

14. Verification and Test Results

14.1 Scheduler result

Test	Observed result
Nodes processed	16
Radio range	500.0 m
Frame length	9 unique timeslots
Schedule	Conflict-free
Execution	Finalized cleanly

14.2 EMANE interfaces

All 16 interfaces emane1 through emane16 were observed in the WSL2 environment.

14.3 IP addressing

The interfaces were assigned 172.30.1.1/24 through 172.30.1.16/24. This matched the per-NEM transport definitions.

14.4 Schedule publication

The command emaneevent-tdmaschedule emane/tdma-schedule.xml completed without an error message in the demonstrated run.

14.5 Packet-level connectivity

Metric	Result
Destination tested	172.30.1.2
Packets transmitted	5
Packets received	5
Packet loss	0%
Average RTT	0.093 ms

The ping result confirms successful packet-level connectivity between the tested EMANE virtual interfaces in the running environment.

15. WSL2 Environment Considerations

During startup, EMANE reported real-time priority warnings such as 'Unable to set Real Time Priority'. These messages are consistent with running the emulation environment under WSL2 where real-time scheduling privileges may be restricted.

The environment also initially reported an inability to open the TUN device. After verifying /dev/net/tun and testing TUN creation with ip tuntap, EMANE was launched with elevated privileges and successfully created emane1 through emane16.

The project's own EMANE README records the same limitation: the demonstration validates schedule loading, TDMA configuration, virtual interfaces, and packet-level communication, but should not be interpreted as real-time wireless hardware validation.

16. Limitations

- Validation was performed in WSL2 rather than on dedicated wireless hardware.
- Real-time thread-priority warnings were observed.
- The 500 m radio range is a configured model parameter.
- Packet-level ping validation is not a substitute for RF measurement.
- The demonstrated schedule is specific to the supplied topology and scheduling assumptions; changing topology or range requires regeneration and revalidation.

17. Future Scope

- Add a graphical topology and schedule visualization.
- Support larger topologies and configurable grid dimensions.
- Add comparative metrics such as slot utilization and schedule length against baseline approaches.
- Automate generation of EMANE schedule XML from the Python scheduler output.
- Add automated regression tests for topology changes and edge cases.
- Provide configurable radio ranges and conflict-model parameters from the command line or a configuration file.
- Extend validation to multiple traffic flows and larger packet workloads.
- Evaluate the scheduler and emulation workflow on a native Linux environment where real-time scheduling can be configured more directly.

18. Conclusion

The TDMA Scheduler project establishes a complete workflow for topology-aware TDMA schedule generation and EMANE-based validation. The demonstrated scheduler processed a 16-node topology with a configured 500 m radio range and generated a conflict-free 9-slot frame. The resulting schedule was represented in EMANE XML and successfully used with a 16-NEM platform.

The integration was verified through creation of 16 virtual EMANE interfaces, assignment of the expected 172.30.1.x addresses, successful TDMA event publication, and a packet-level connectivity test showing 5 packets transmitted, 5 received, and 0% packet loss.

Within the stated WSL2 limitations, these results demonstrate that the scheduling algorithm, TDMA configuration, EMANE integration, virtual networking, and basic packet-level validation operate together as an end-to-end prototype.

19. Appendix — Commands and Reference Tables

19.1 Core verification commands

```
emane --version
python3 run_from_file.py sample_input.json --range 500
emaneevent-tdmaschedule emane/tdma-schedule.xml
ip -br link | grep emane
ip -br addr | grep emane
ping -c 5 172.30.1.2
```

19.2 Key EMANE paths

Path	Purpose
/usr/local/bin/emane	EMANE executable
/usr/local/bin/emaneevent-tdmaschedule	TDMA event publisher
/usr/local/lib/libemane.so	Core EMANE shared library
/usr/local/lib/libtdmaeventschedulerradiomodel.so	TDMA radio-model library
/usr/local/share/emane/dtd	EMANE DTD definitions
/usr/local/share/emane/xml/models/mac/tdmaeventscheduler	TDMA model XML resources
/usr/local/lib/python3/dist-packages/emane	EMANE Python package

19.3 Final project verification checklist

Checkpoint	Status
Python scheduler executes	Verified
16 nodes processed	Verified
9-slot schedule generated	Verified
Conflict-free schedule reported	Verified
EMANE 1.5.3 installed	Verified
TDMA event utility works	Verified
16 NEM interfaces created	Verified
172.30.1.x addresses assigned	Verified
TDMA schedule published	Verified
5/5 packet connectivity	Verified
0% packet loss	Verified

End of Document